



Wrocław University of Technology

AMQP & Virtualization + Cloud Computing

Tomasz Walkowiak



Data formats

- XML
- JSON (still costly)
- Protocol Buffers
 - IDL, binary, Google
 - 7-10x faster than JSON
 - Compilation to (.proto) classes
 - Java, C++, C#, Python
- Other (CSV 😊)

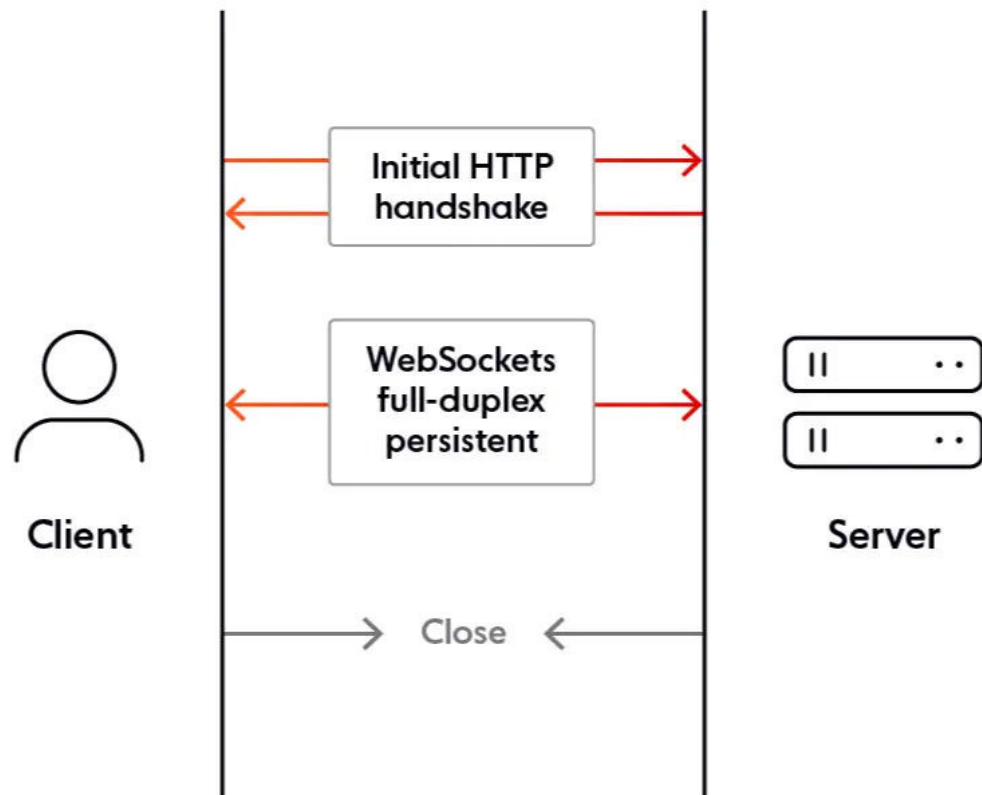
```
message Person {  
    required string name = 1;  
    required int32 id = 2;  
    optional string email = 3;
```

```
enum PhoneType {  
    MOBILE = 0;  
    HOME = 1;  
    WORK = 2;  
}
```

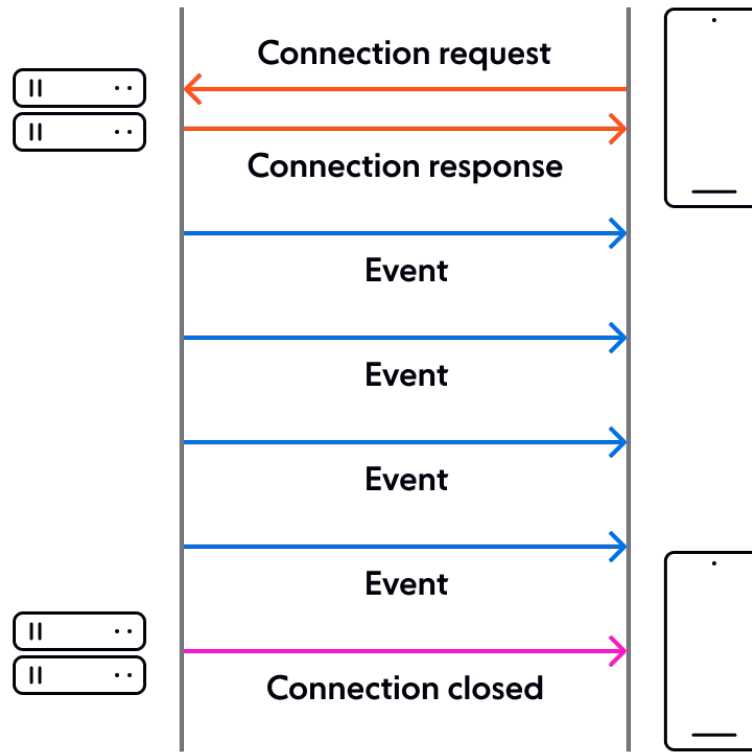
Bardziej skomplikowane schematy komunikacji

- Classic HTTP is unidirectional
 - Always the client initiates the communication
 - And the server responds
- How to make communications from the server:
 - polling (we ask every so often)
 - long-polling (we use timeout)
 - WebSocket (two-way communication)
 - SSE (it's not two-way, but ...)

WebSocket



Server-Sent Events





REST problems (recap)

- Addresses of endpoints:
 - `http://156.17.43.12:32322/whatever/what`
 - Solution:
 - Central registry
 - Central endpoint: proxy
 - DNS, but ports ?
 - Lets' make each service working on different IP
 - Other protocols, e.g. AMQP
- Synchronous HTTP/HTTP:
 - timeouty (default 30s), cascade of errors
 - Long running services
 - Callback (ie.. **Asynchronous Server API JAX-RS**)
- Solution asynchronity
 - WebSockets
 - AMQP - Advanced Message Queuing Protocol

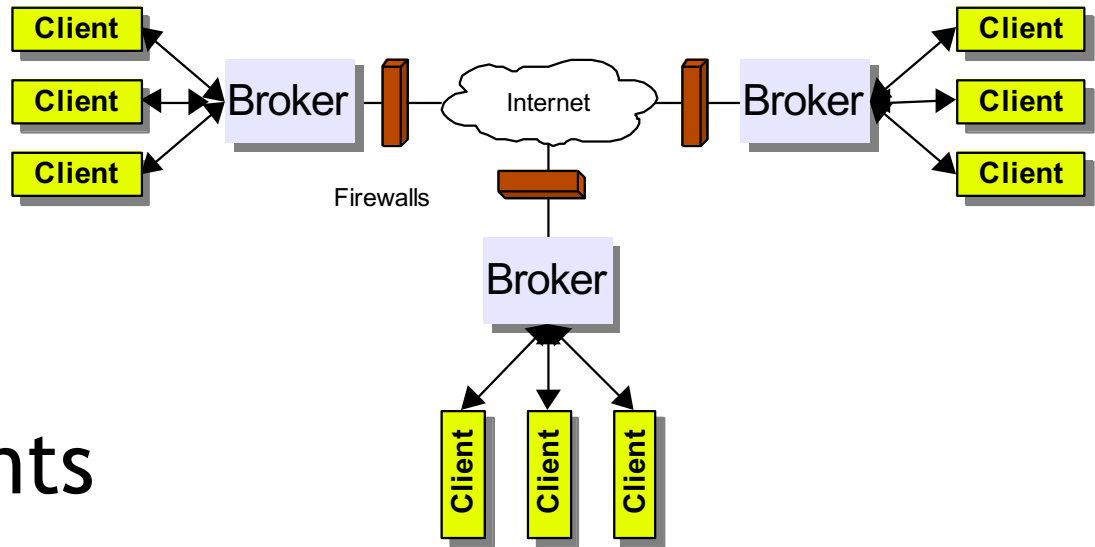


AMQP

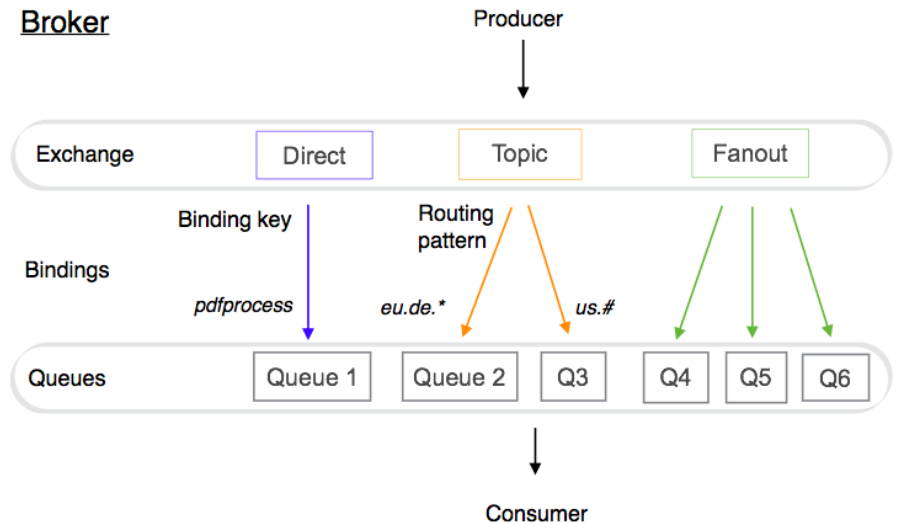
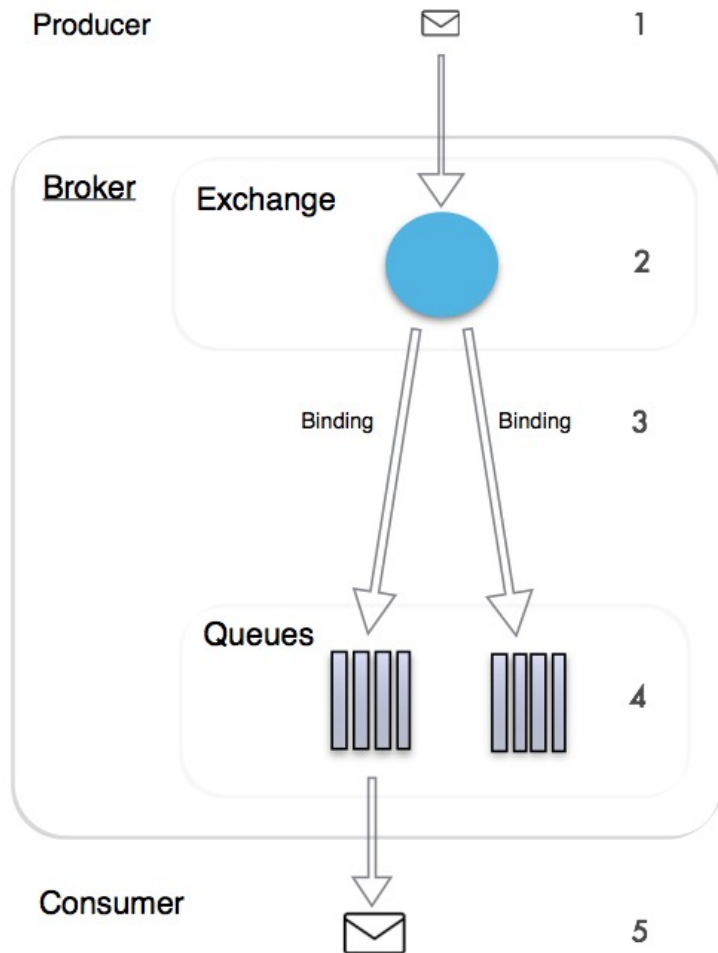
- message-oriented middleware
- Binary protocol over TCP/IP
- We have JMS (API from JavaEE)
- Implementations:
 - Storm MQ
 - Apache ActiveMQ
 - Rabbit MQ

RabbitMQ

- Fast, Erlang,
- Broker
 - connection
 - channels
 - queues
- Different clients
 - C++, C#, Python, Java, PHP, JS



Communion schema



Point-Point



```
import pika
```

```
connection = pika.BlockingConnection(pika.ConnectionParameters(  
    host='localhost'))
```

```
channel = connection.channel()
```

```
channel.queue_declare(queue='hello')
```

```
def callback(ch, method, properties, body):
```

```
    print(" [x] Received %r" % body)
```

```
channel.basic_consume(callback,
```

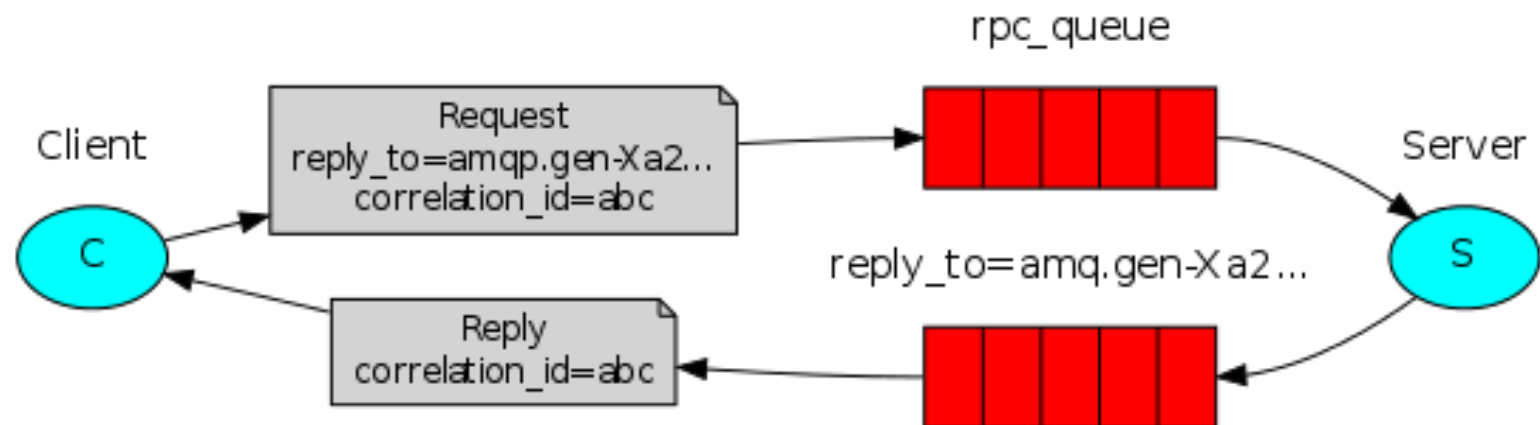
```
    queue='hello',
```

```
    no_ack=True)
```

```
print(' [*] Waiting for messages. To exit press CTRL+C')
```

```
channel.start_consuming()
```

RPC





RPC -server

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost");
Connection connection = factory.newConnection();
Channel channel = connection.createChannel();
channel.queueDeclare(RPC_QUEUE_NAME, false, false, false, null);
channel.basicQos(1);
Consumer consumer = new DefaultConsumer(channel) {    @Override
public void handleDelivery(String consumerTag, Envelope envelope,
AMQP.BasicProperties properties, byte[] body) throws IOException
{ AMQP.BasicProperties replyProps = new
AMQP.BasicProperties.Builder().correlationId(properties.getCorrelationId()).build();
String response = „....“;
channel.basicPublish( "", properties.getReplyTo(), replyProps, response.getBytes("UTF-
8"));
channel.basicAck(envelope.getDeliveryTag(), false);
}
};
channel.basicConsume(RPC_QUEUE_NAME, false, consumer);
```



RPC - client

```
class CL(object):
    def __init__(self):
        ....
        result = self.channel.queue_declare(exclusive=True)
        self.callback_queue = result.method.queue
    def on_response(self, ch, method, props, body):
        if self.corr_id == props.correlation_id:
            self.response = body

    def call(self, n):
        self.response = None; self.corr_id = str(uuid.uuid4())
        self.channel.basic_publish(exchange="", routing_key='rpc_queue',
                                   properties=pika.BasicProperties(reply_to = self.callback_queue,
                                                                    correlation_id = self.corr_id ...
        while self.response is None:
            self.connection.process_data_events()
        return self.response
```



gRPC

- classic RPC and streaming RPC
- Multi-language: **Java, Go, C, C++, Node.js, Python, Ruby, Objective-C, PHP and C#**
- IDL: **Protocol Buffers 3, Flatbuffers**
- Transport: **HTTP2**
- Auth: **SSL/TLS**
- Open sourced: **github.com/grpc/**

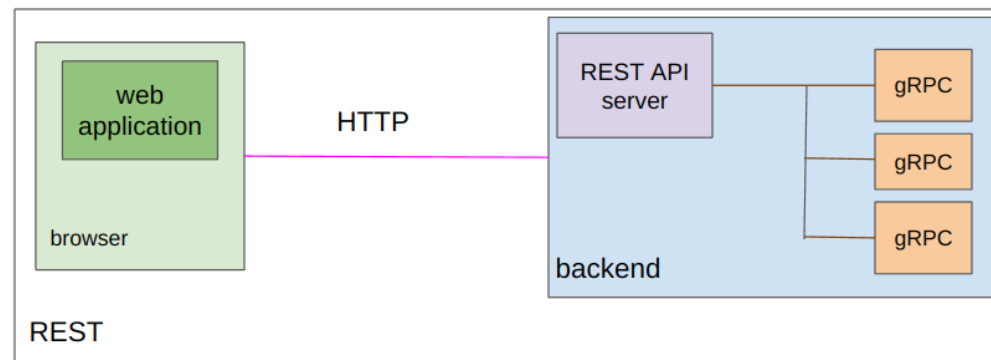


HTTP 2.0 (2015), HTTP/1.1 plus:

- Binary Framing Layer -request/response is divided into small messages and framed in binary format, making message transmission efficient; request/response multiplexing possible without blocking network resources.
- Streaming - Bidirectional full-duplex streaming in which the client can request and the server can respond simultaneously.
- Flow Control detailed control of memory used to buffer in-flight messages.
- Header Compression - Everything in HTTP/2, including headers, is encoded before sending, significantly improving overall performance. Using the HPACK

gRPC

- Protobuf + HTTP 2
- so problems with browsers



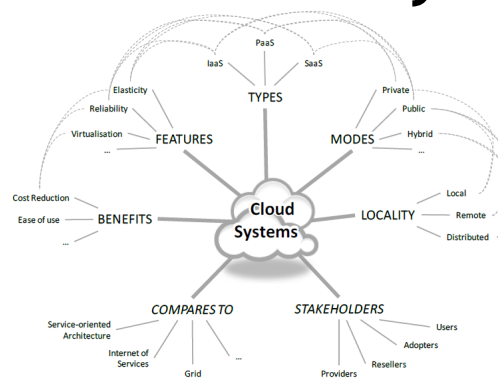
- But:
- supports both synchronous and asynchronous processing, which can be used to perform different types of interaction and streaming RPCs.

Cloud Computing: The Next „New Thing”

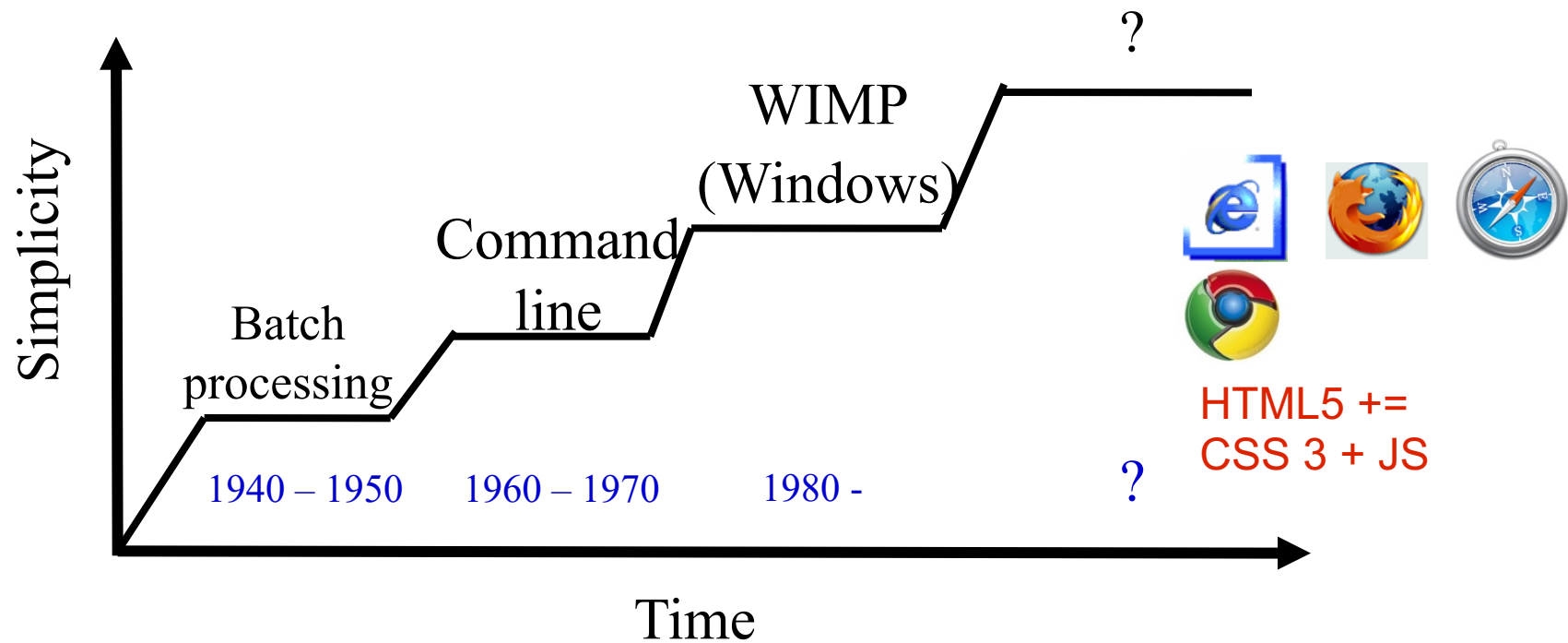
- A general term for the style of computing that supports web services, search, social networking
- Increasingly powerful and universal
- Enables a new kind of massively scaled, elastic app



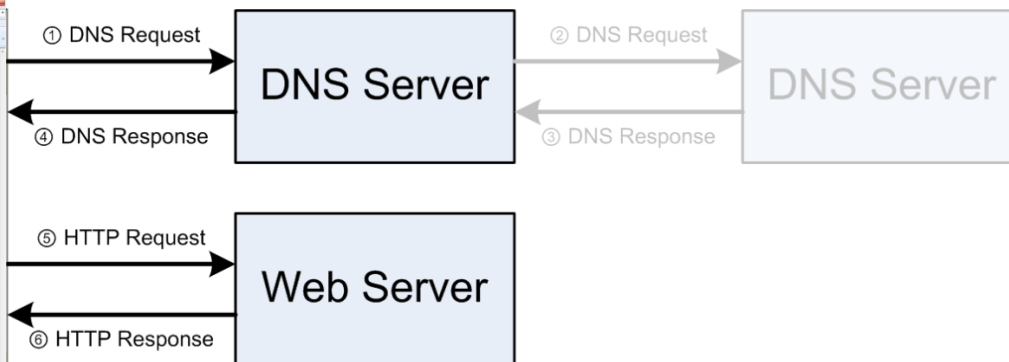
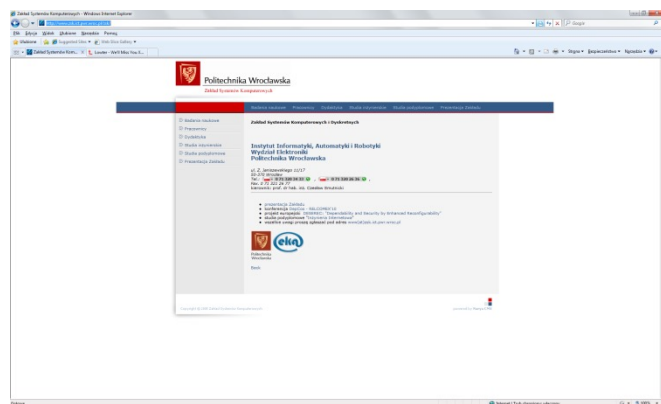
In my country of Khazackstan, many excellent hacker. We hack cloud, steal private stuff of whole world!



Computer user interface



Picture of Web App



Browser:

- A kind of mini-operating system
- Web pages are the programs
- they can contain real executable code)

IT Technology Generations



**Central
Systems**



**Client-Server
PC - 1981**



**Internet
Biznes on-line**



**Mobile Technologies
Cloud Computing
Analytics**



NIST definition

„**Cloud computing** is a model for enabling ubiquitous, convenient, **on-demand network access** to a shared pool of **configurable computing resources** (e.g. networks, servers, storage, applications, and services) that can be rapidly **provisioned** and released with **minimal management effort** or service provider interaction.”



Clouds are hosted by data centers

- Huge data centers, far larger than past systems
- Very automated: far from where developers work. Often close to where power is generated
- Packed for high efficiency. Each machine hosts many applications (usually in lightweight virtual machines to provide isolation)
- Scheduled to keep everything busy (but overloads hurt performance so we avoid them)

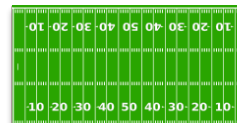


Clouds are cheaper... and winning...

Range in size from “edge” facilities to megascale.

Incredible economies of scale

Approximate costs for a small size center (1K servers) and a larger, 50K server center.



Each data center is
11.5 times
the size of a football field

Technology	Cost in small-sized Data Center	Cost in Large Data Center	Cloud Advantage
Network	\$95 per Mbps/month	\$13 per Mbps/month	7.1
Storage	\$2.20 per GB/month	\$0.40 per GB/month	5.7
Administration	~140 servers/Administrator	>1000 Servers/Administrator	7.1





Key benefits?

- Machines busier, earn more \$'s for each \$ investment
 - Hardware handled a whole truckload at a time
- Applications far more standardized
 - Automated management: few “sys admins” needed
 - Power consumed near generator: less wastage
 - Data center runs hot, wasting less on cooling
 - Can “rent” resources rather than owning them
- Supports new, extremely large-scale services
 - Elasticity to accomodate surging demands
 - Can accumulate and access massive amounts of data
 - But must read or process it in a massively parallel way
 - Enables overnight emergence of major companies, but scalability model does require **new programming** styles, and imposes **new limits**

Cloud computing is enabled by

Virtualized

Higher utilization
Economy of scale
benefits
Lower capital expense
Lower operating expense



Doing more with less

Standardized

Easier access
Flexible pricing
Reuse and share
Easier to integrate



Higher quality services

Automated

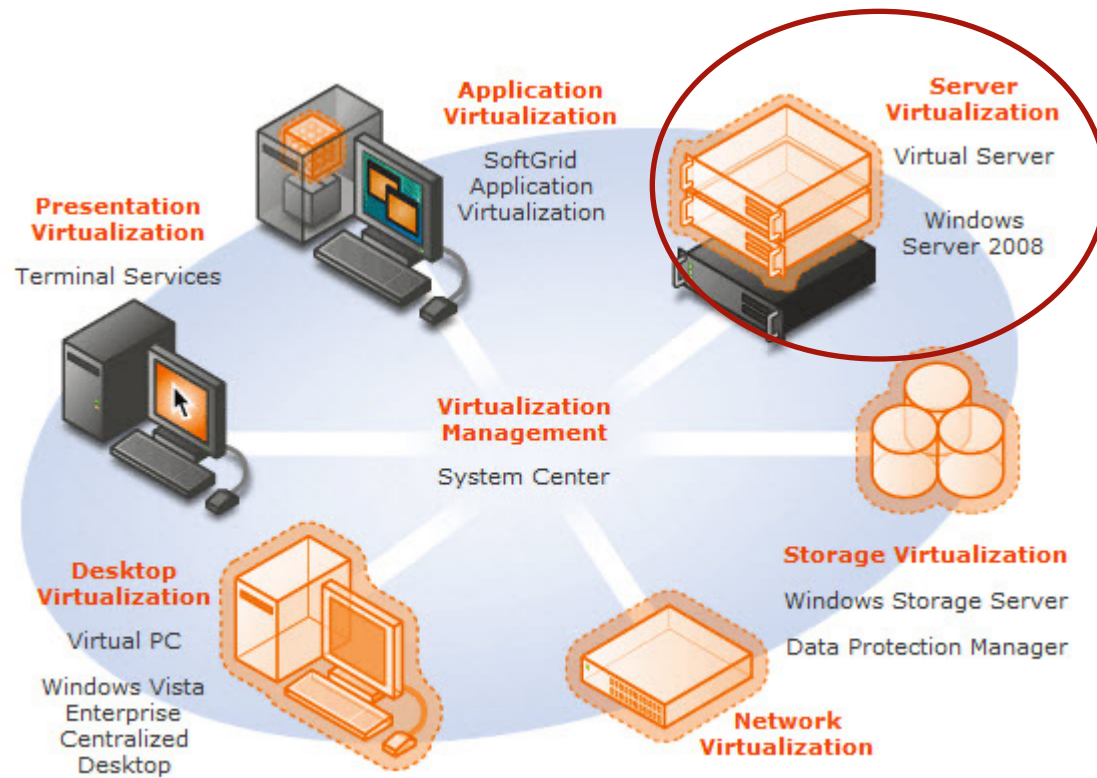
Faster cycle times
Lower support costs
Optimized utilization
Improved compliance
Optimized security
End user experience



Breakthrough agility and
reducing risk

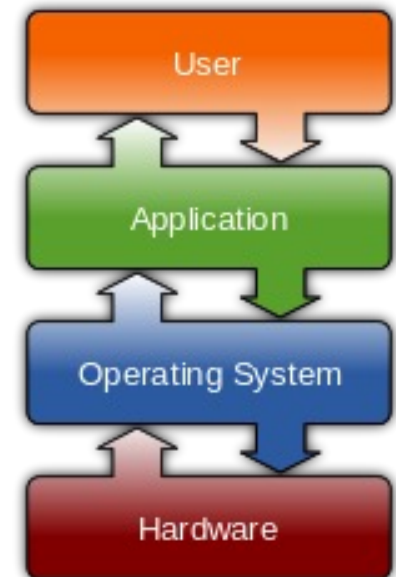


Virtualization - technique behind CC



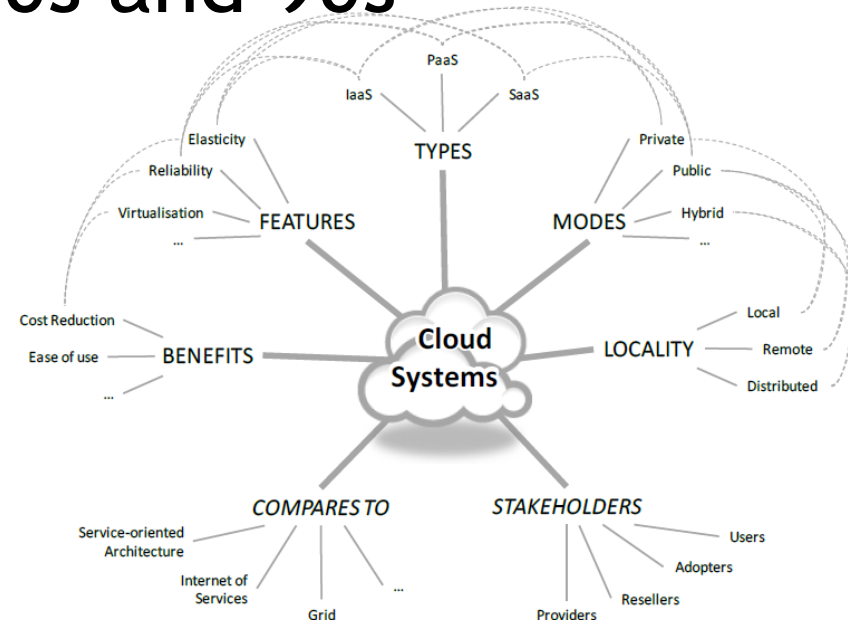
Server virtualization

Methodology of dividing the computer (or server) resources into multiple separate execution environments called virtual machines



History

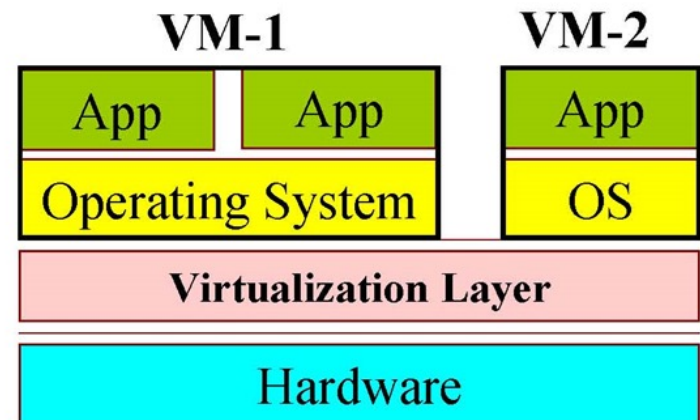
- Introduced by IBM in the 1960s
 - To boost utilization of large, expensive mainframe systems
- Gave away to C/S in 80s and 90s
- Become hot again
 - Servers are cheap and powerful
 - Become the key component of cloud computing





Characteristics of Virtualization

- Established by Popek and Goldberg in 1974
 - **Fidelity**: Software on the Virtual Machine Monitor executes identically to its execution on hardware
 - **Performance**: The majority of instructions are executed by the hardware without the VMM intervention
 - **Safety**: The VMM manages all hardware resources

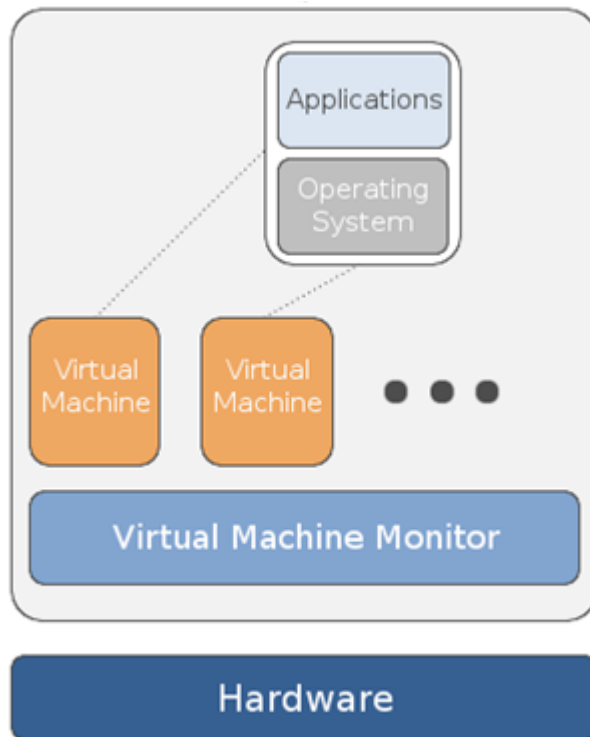




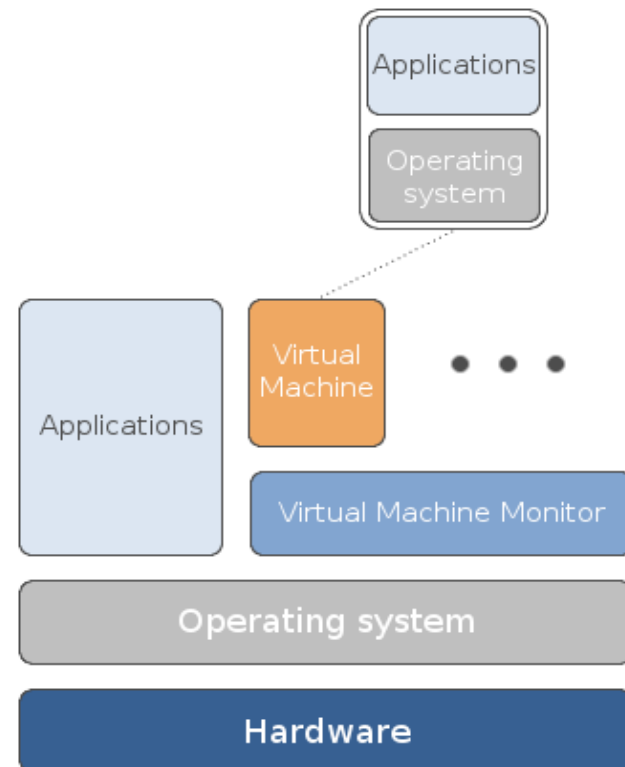
Hypervisor (Virtual Machine Monitor)

- Software providing the abstraction of a virtual machines
- Manage multiple operating systems and to provide resources which each OS requires
- Two hypervisor types can be distinguished:
 - Type 1
 - Type 2

Hypervisor types



Type 1 Hypervisor
VMware Vsphere, Xen



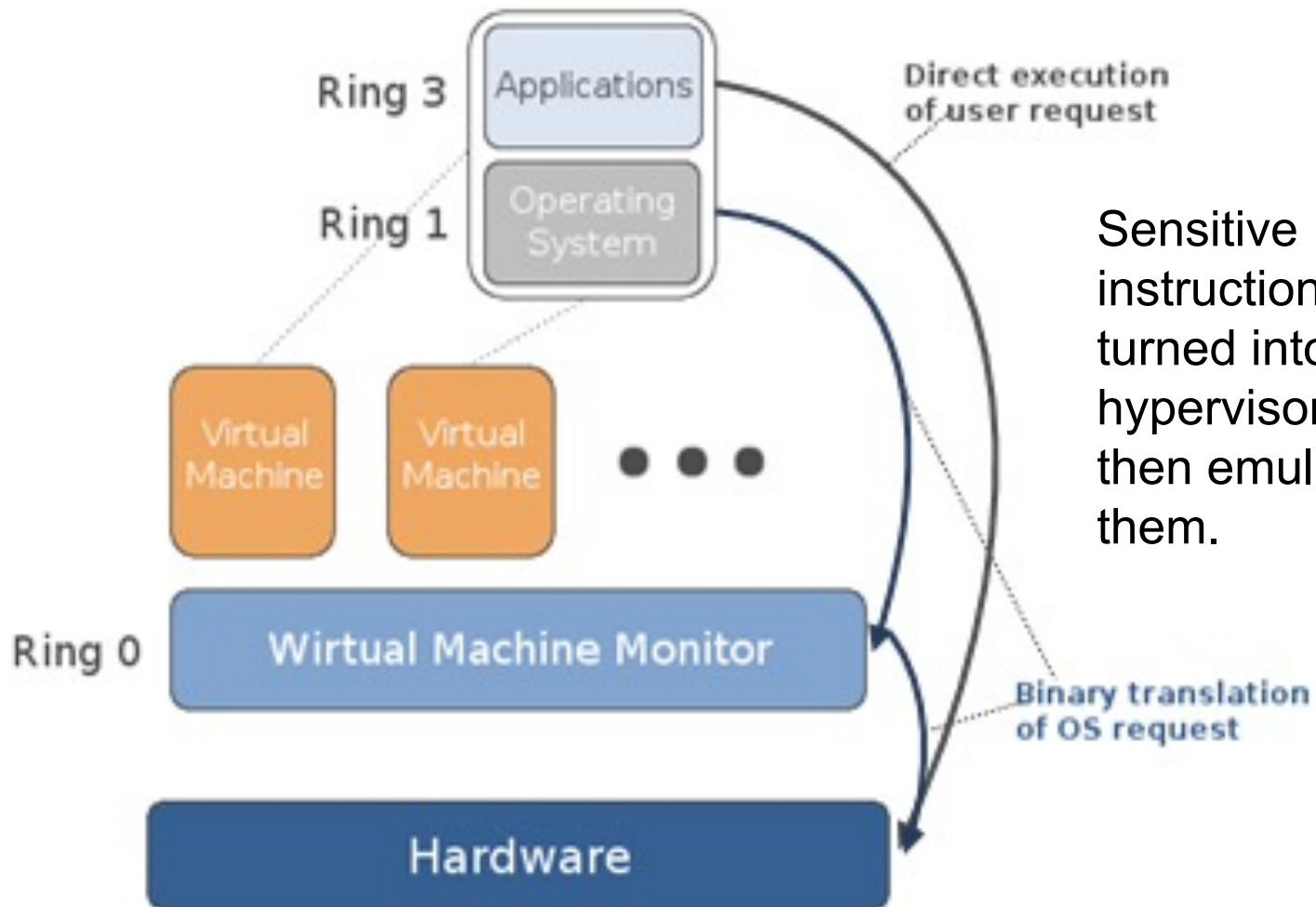
Type 2 Hypervisor
VMware Works., VirtualBox



Techniques for CPU Virtualization

- Full virtualization using **binary translation**
 - VMWare vSphere
- OS assisted virtualization or **Para-virtualization**
 - Xen
- **Hardware-assisted virtualization**
Virtualization in CPU
 - Intel VT-x

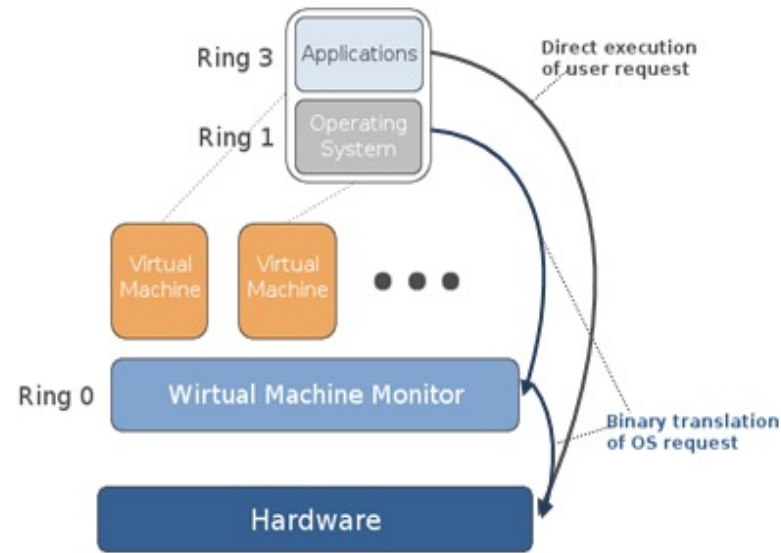
Technique 1- Full Virtualization using Binary Translation



Sensitive instructions are turned into calls to hypervisor which then emulates them.

Full Virtualization

- All **sensitive instructions** are replaced by calls to procedures that **emulate** these instructions
 - No sensitive instructions issued by guest OS are ever executed by true hardware
 - Sensitive instructions are caught and replaced with a call to a VMM procedure(s) that handle it.
 - This technique is called **binary translation**.

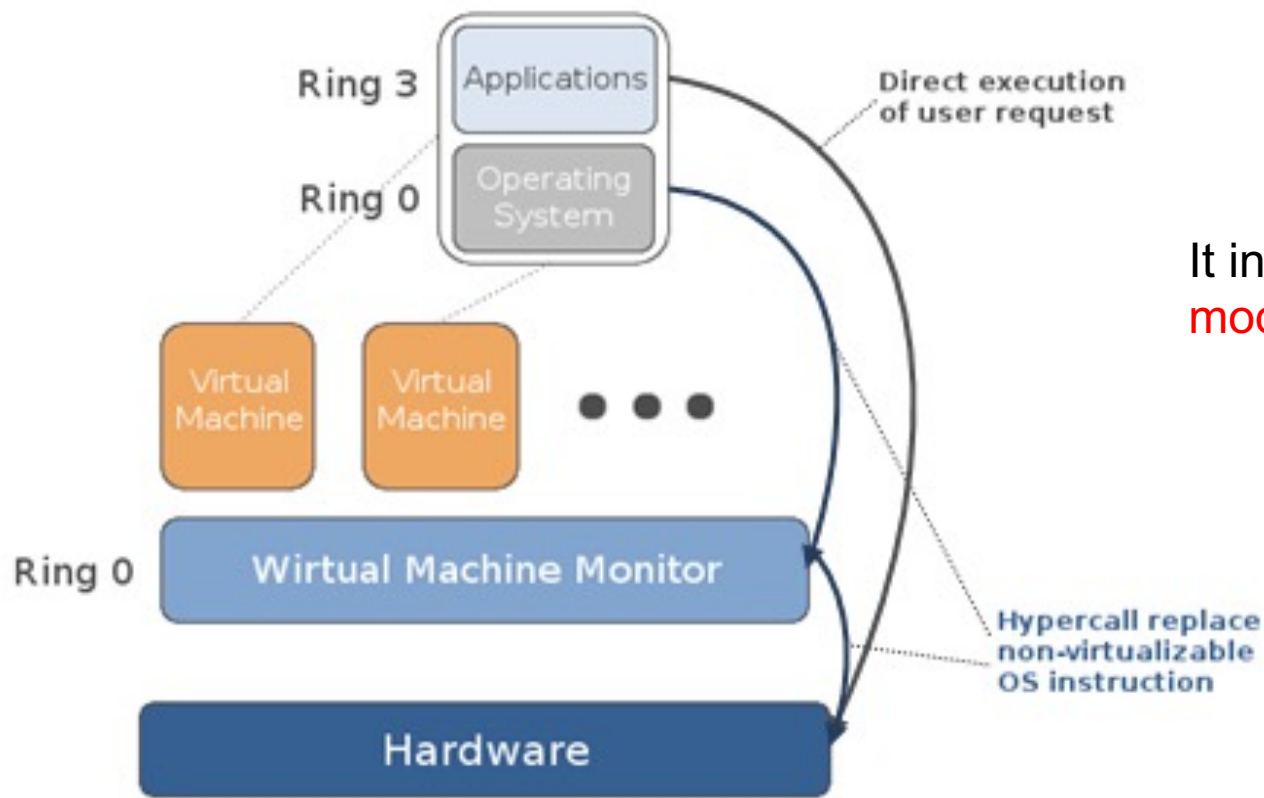




Full Virtualization - example: VMware

- Combination of binary translation and direct execution techniques
- Translation of kernel code to replace non-virtualizable instructions with new sequences of instructions that have intended effect on virtual hardware
- User level code is directly executed on processor for high performance virtualization
- Guest OS does not need any modification
- No need for any hardware assist or OS assist to virtualize sensitive and privileged instructions
- Hypervisor translates all OS instructions on fly and caches results for future use while user level instructions run unmodified at native speed.

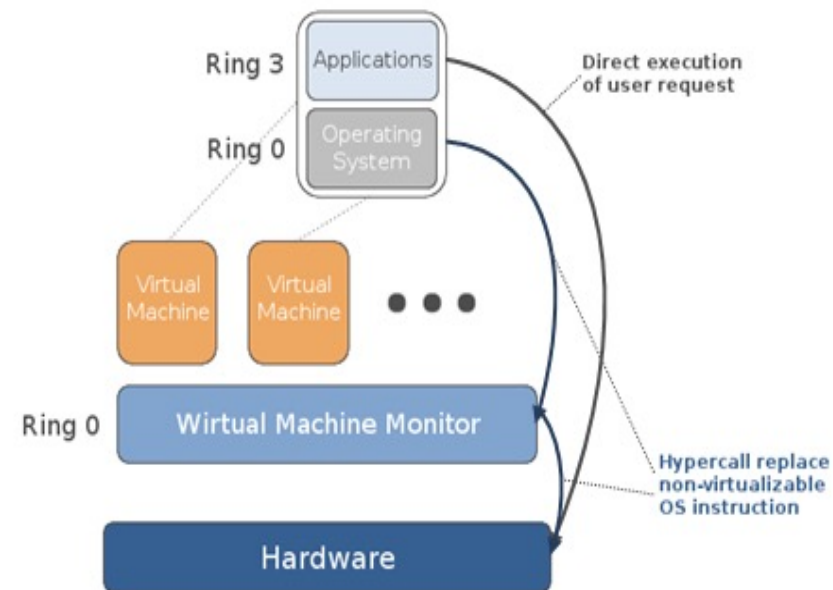
Technique 2 - OS Assisted Virtualization or Para-virtualization



It involves
modifying OS kernel

Technique 2 - Para-virtualization

- Paravirtualization refers to **communication between guest OS and hypervisor** to improve performance and efficiency
- It involves **modifying OS kernel** to replace non-virtualizable instructions with hypercalls that communicate directly with virtualization layer hypervisor
- Hypervisor provides **APIs** to critical kernel operations
 - Memory management, interrupt handling, time keeping

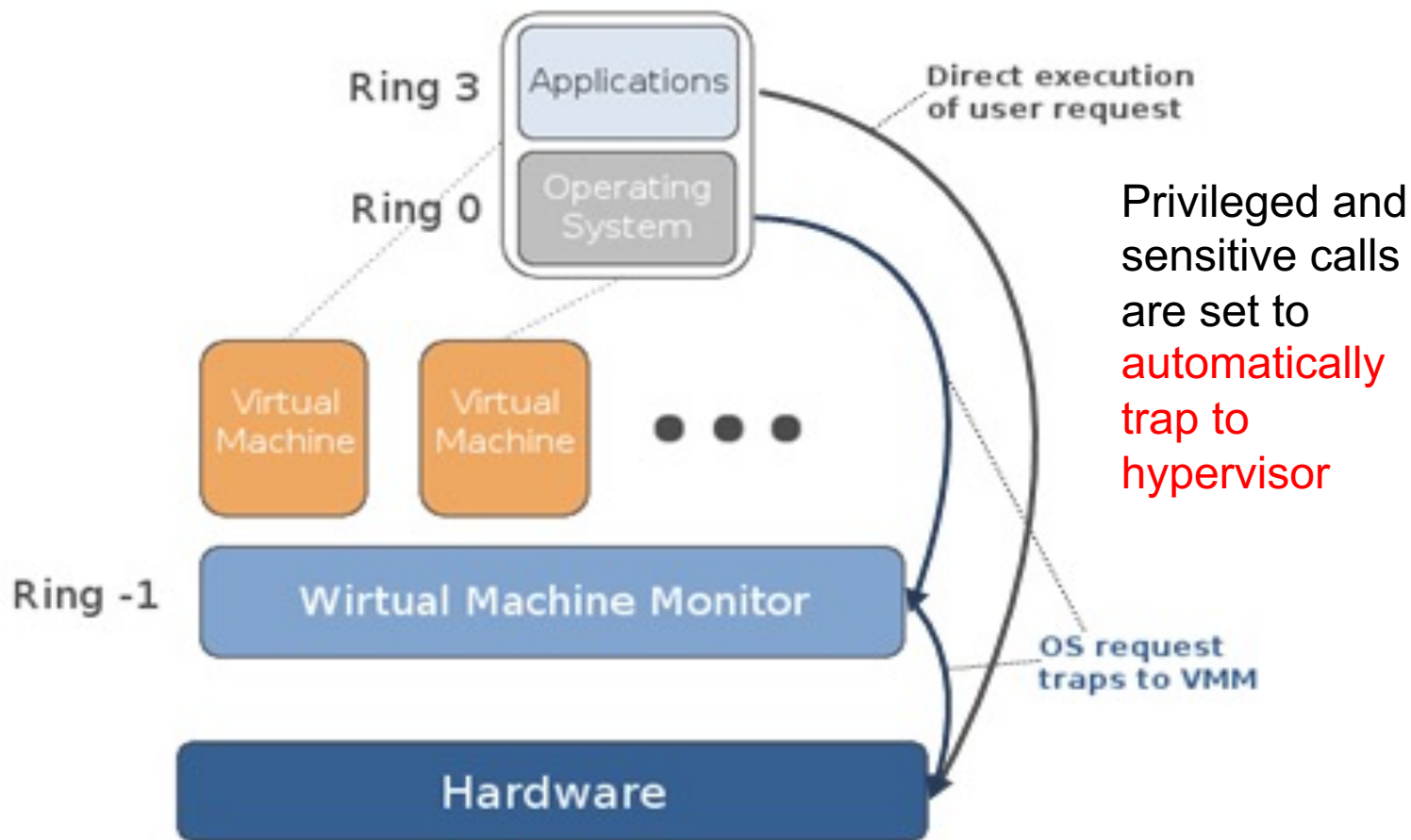


Paravirtualization

- Value of paravirtualization is in **lower virtualization overhead**
- Performance advantage of paravirtualization over full virtualization varies greatly depending on workload
- Its **compatibility and portability is poor**
- It introduces **significant support and maintainability issues** in production
 - Requires deep OS kernel modifications
- **Xen** - open source project:
 - Virtualizes processor and memory using modified Linux kernel
 - Virtualizes I/O using custom guest OS device drivers



Technique 3 - Hardware Assisted Virtualization





Hardware-assisted Virtualization

- Hardware vendors develop new features to **simplify virtualization techniques**
 - Intel Virtualization Technology (VT-x)
 - AMD's AMD-V
 - Both target privileged instructions with **new CPU execution mode feature** that allows VMM to run in new root mode below Ring 0
- Privileged and sensitive calls are set to **automatically trap to hypervisor**, removing need for either binary translation or paravirtualization

Virtualization - summary

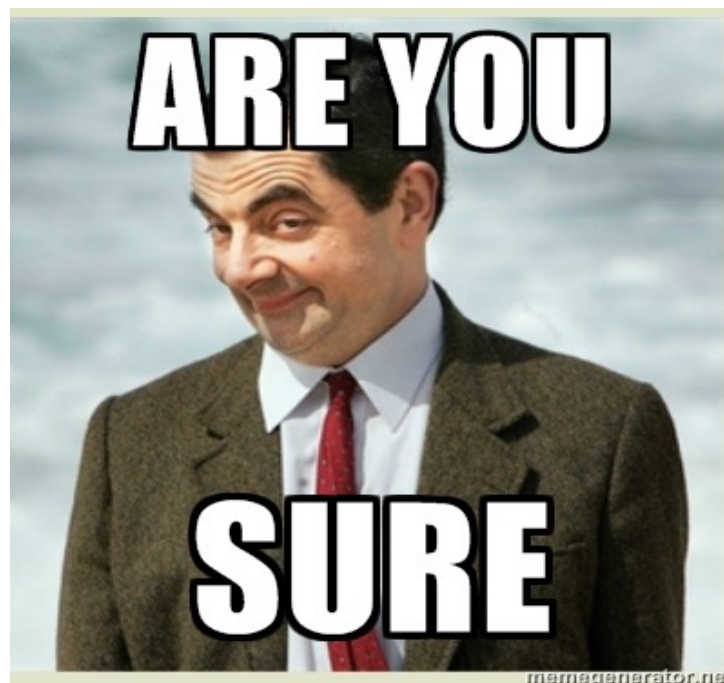
- Host operating system that provides an abstraction layer for running virtual “guest” operating systems
 - “hypervisor” or “virtual machine monitor”
- Enables guest OSs to run in isolation of other OSs
- Run multiple types of OSs
 - Increases utilization of physical servers
 - Enables portability of virtual servers between physical servers
- Makes administrator live easier



Security of virtualization



Virtualization is considered by many as secure technology



Threats division

- Human mistakes
- Malware-based
- Failures





Human mistakes assessment

Threat

Inadequate protection

VM sprawl

Misconfiguration

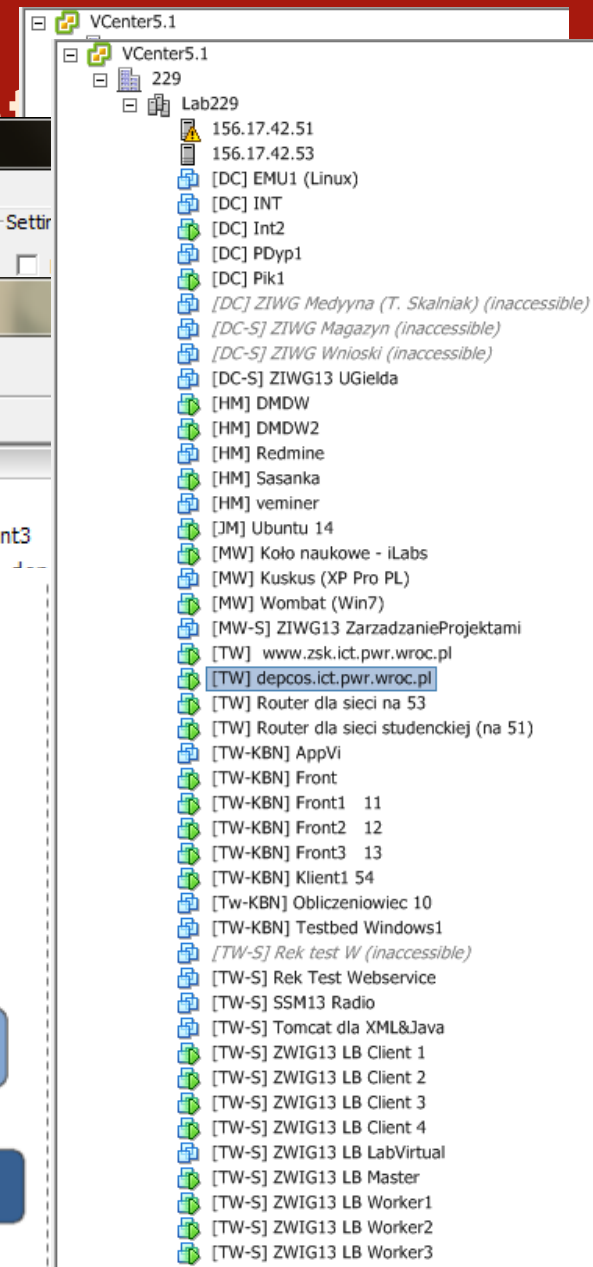
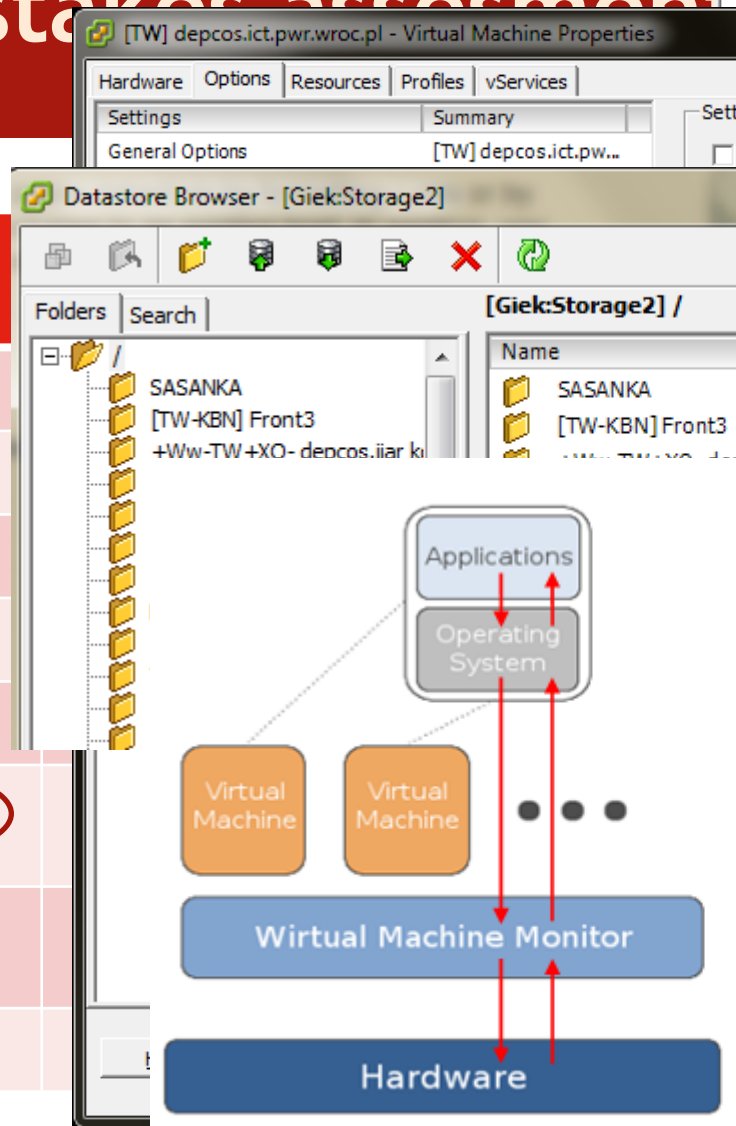
VM rollback

Dormant VM

Files deletion in virtual file system

Damage of virtual disk files

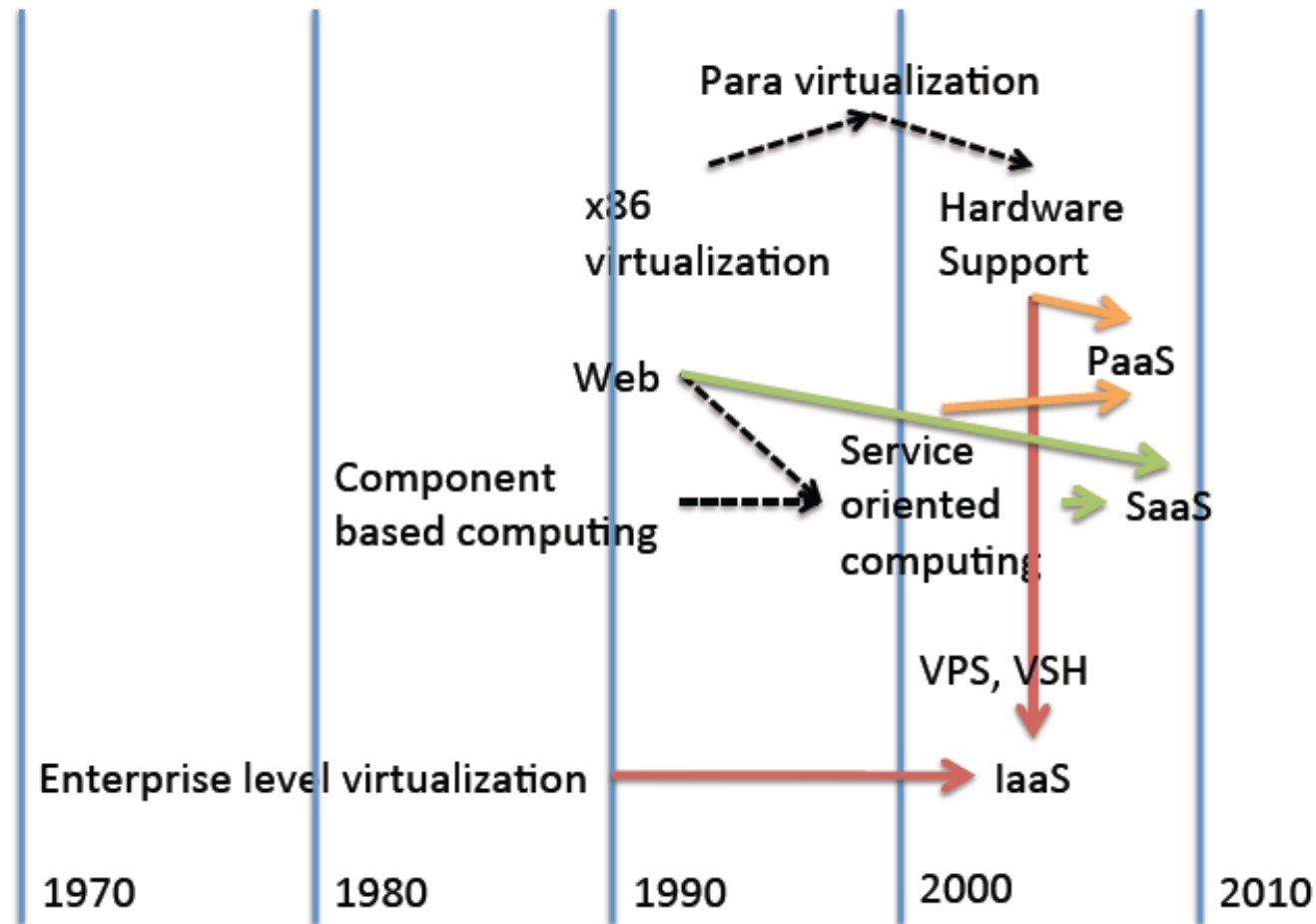
Loss of performance



Risk in server virtualization

- The most dangerous identified threads are connected with human mistakes:
 - inadequate protection,
 - misconfiguration,
- The second level of importance includes:
 - VM escape, VM hopping, VMware vCenter compromising,
 - abuse of privileges,
 - virtual disk file/disk failure.

Evolution of Cloud technologies



Legend: PaaS – Platform as a Service, SaaS – Software as a Service, VPS – Virtual private server, VSH - Virtual server hosting, IaaS – Infrastructure as a Service

CC characteristics

- **Virtual**
 - software, databases, Web servers, operating systems, storage and networking as virtual servers.
- **On demand**
 - add and subtract processors, memory, network bandwidth, storage
- Cloud computing often leverages:
 - Massive scale
 - Free software
 - Autonomic computing
 - Multi-tenancy
 - Geographically distributed systems
 - Advanced security technologies
- **Pay-as-you-go model**



Acronyms! (How to be a party bore)

- One issue with the cloud is that it has a million acronyms: IaaS, SaaS, PaaS, SOAP, AWS, EC2, S3...
 - These make for a *very* confusing landscape!
 - But a business perspective on the cloud only needs to focus on a few of them, as a starting point
- What does the “aaS” mean?
 - Cloud vendors sell “services”
 - “aaS” == “as a Service”

The Important *aaS options

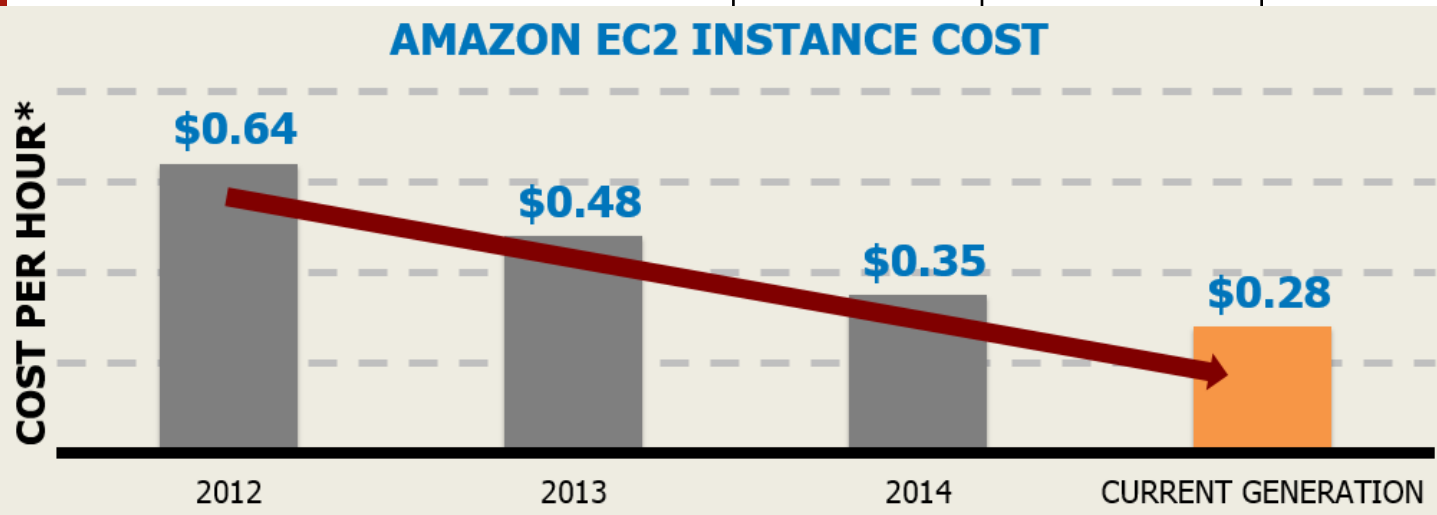
- Infrastructure. (IaaS: Infrastructure as a Service)
 - Cloud vendor rents you some hardware
 - A network, perhaps a wide-area network
 - A machine, always “virtual” but perhaps just for you
 - A file server, again virtual, but you can save files in it
 - They operate this for you, and you pay for what you think you need (or sometimes, for what you use)
 - And they sell backup services too
- For example, you could rent a private Internet from AT&T, or 500 computers from Amazon EC2
 - AWS is elastic: you rent and pay by the hour
 - AWS can accommodate huge swings in your needs





Amazon EC2 pricing (2018)

Machine	Memory (GB)	Compute Units (ECU)	Local Storage (GB)	Cost / hour
t1.micro	0.615	2	0	\$0.02
m1.xlarge	15	8	1680	\$0.48
cc2.8xlarge	60.5	88	3360	\$2.40



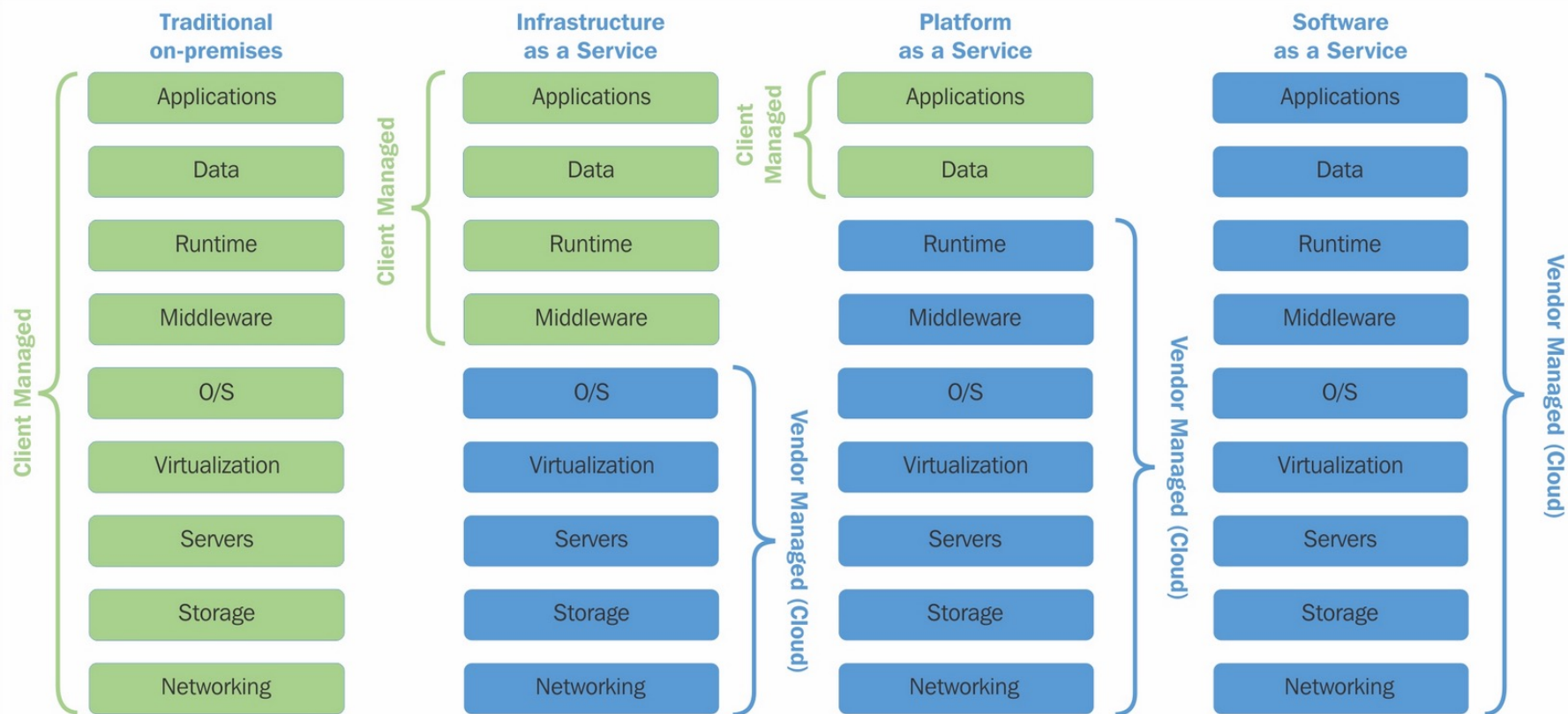


aws.amazon.com/ec2/pricing/on-demand/

vCPU	ECU	Memory (GiB)	(GB)	2020 Per h	2022
General Purpose - Current Generation					
t2.micro	1	Variable	1	\$0.014	\$0.0116
t2.small	1	Variable	2	\$0.027	\$0.023
t2.medium	2	Variable	4	\$0.054	
t2.2xlarge	8	Variable	32	\$0.432	
m4.large	2	6.5	8	\$0.129	\$0.10
m4.xlarge	4	13	16	\$0.257	
m4.2xlarge	8	26	32	\$0.513	
m4.4xlarge	16	53.5	64	\$1.026	
m4.10xlarge	40	124.5	160	\$2.565	
m4.16xlarge	64	188	256	\$4.104	\$3.20



What is cloud computing?



The Important *aaS options

- Software. (SaaS: Software as a Service)
 - Cloud vendor runs some software that you use remotely
 - Classic example: Salesforce.com has a sophisticated infrastructure that manages your sales contact data
- In effect you “outsource” your sales support system and Salesforce.com runs it for you
 - Other SaaS options: accounting, billing document handling, shared files...
 - They also apply patches, fix bugs...



The Important *aaS options



- Platform. (PaaS: Platform as a Service)
 - Cloud vendor creates a sophisticated platform (typically a software environment for some style of computing, or for database applications)
 - Your folks use it to create a custom solution
 - Cloud vendor runs your solution in an elastic way
- They promise that if you use their PaaS solution, you'll benefit from better scalability, performance, ease of development or other advantages

The Important *aaS options

• Platform. (PaaS: Platform as a Service)



Force.com

- Apex
- Force.com (Equinox)



Google Apps Engine

- Python
- Google



Microsoft Azure

- .Net
- Microsoft



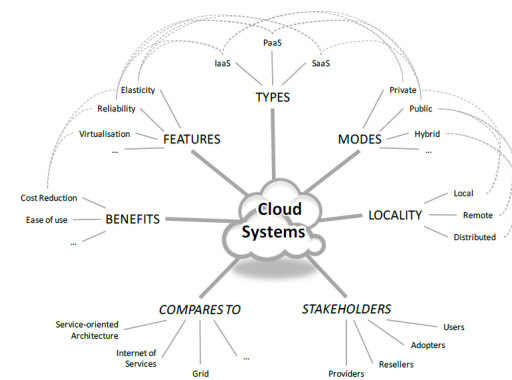
Heroku

- Ruby, Java, Python, Node
- AWS



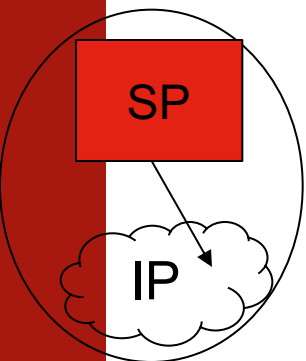
... these aren't the whole cloud

- The cloud *mixes* many models
 - Some integrate humans into the loop, such as outsourced audio-to-text
 - There are companies with specialized roles
 - Akamai: The most famous data hosting company, especially successful for storing videos and images that are used in your web pages. They specialize in rapid data delivery
 - DoubleClick: You leave a frame on your web page, they put the perfect advertisement for this particular user in it

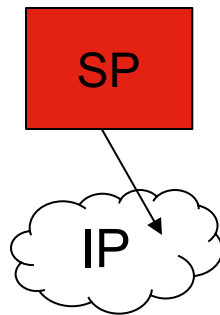


Deployment models

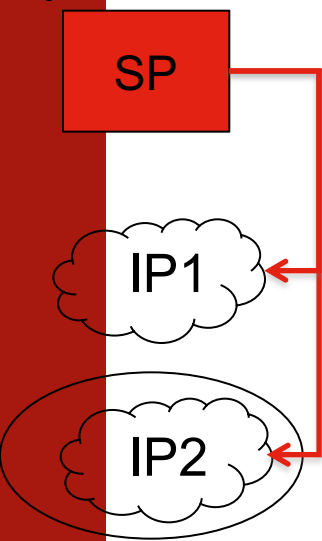
Private Cloud



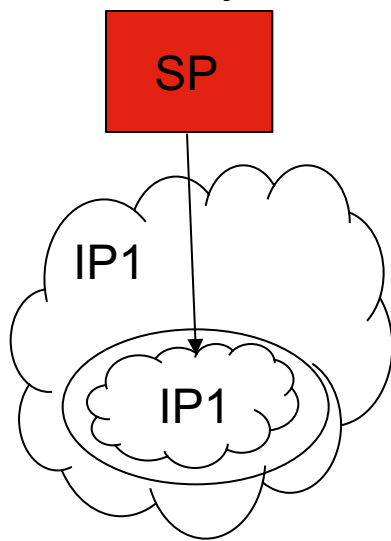
Public Cloud



Hybrid Cloud



Community Cloud



- **Private Clouds**

- Typically owned/leased by the respective enterprise.
- Functionalities are not directly exposed to the customer
- Similar to SaaS from the customer point of view.

- **Public Clouds**

- offer services to users outside of the company.
- Example: Amazon, Google Apps, Windows Azure.

- **Hybrid Clouds**

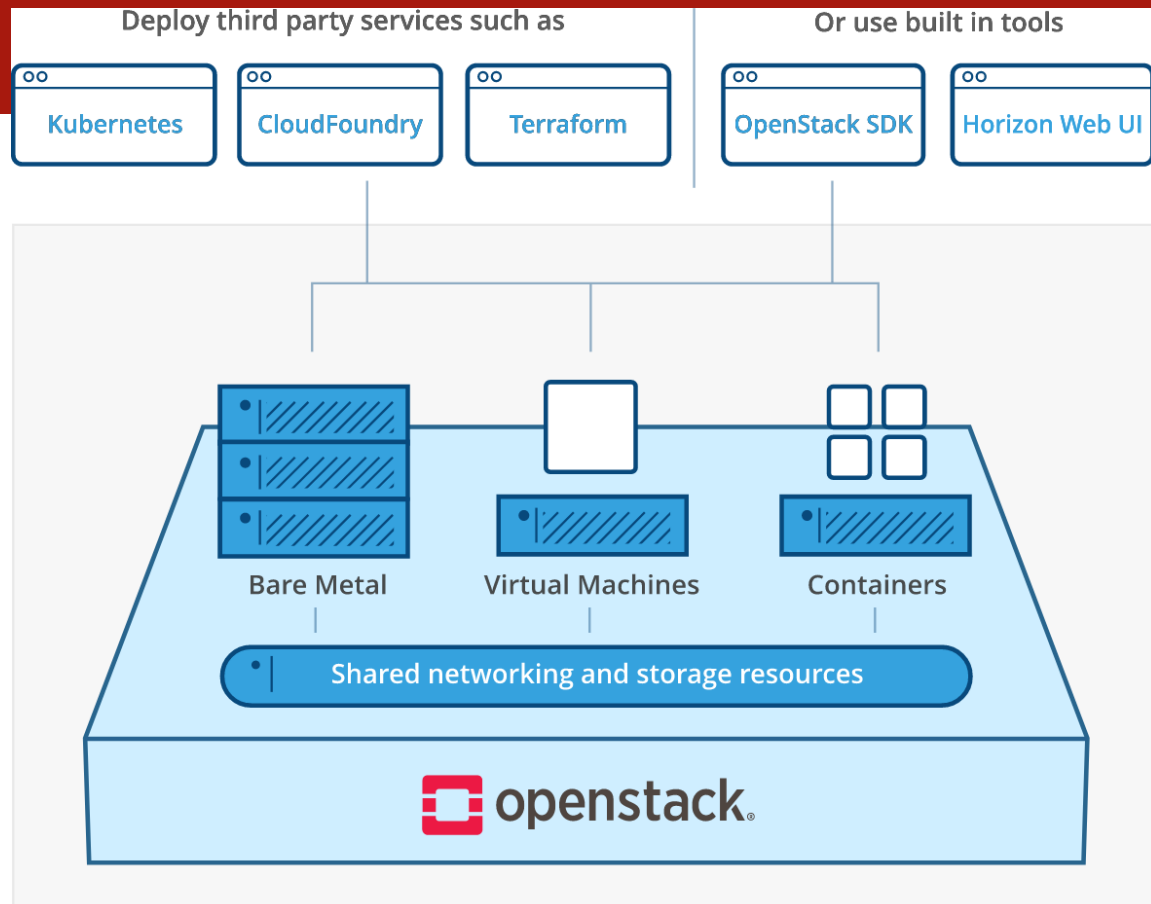
- Consist of a mixed employment of *private* and *public cloud* infrastructures
- Maintains the desired degree of control e.g. over sensitive data

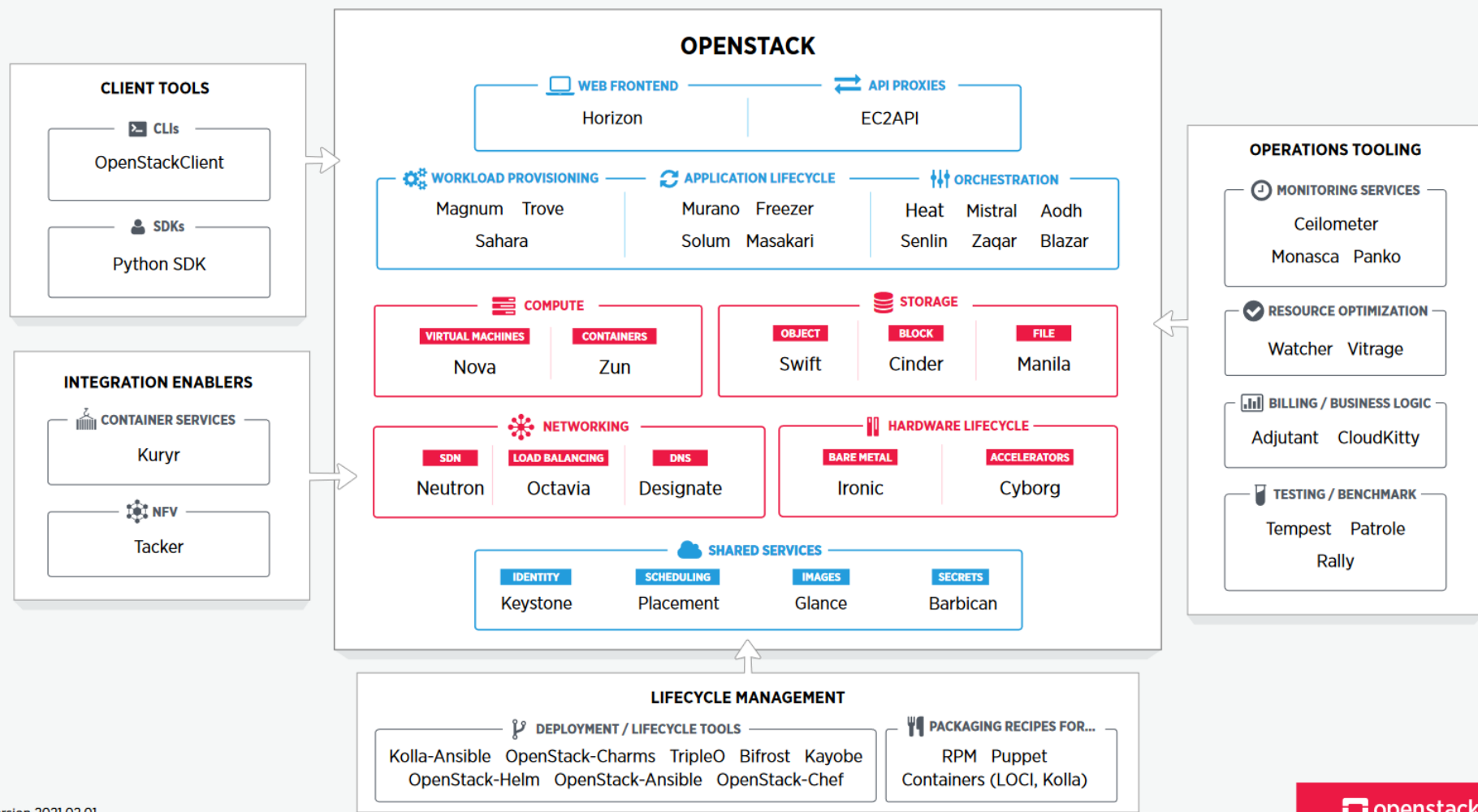
- **Community Clouds**

- A cloud provider resells the infrastructure of others
- Either aggregates public clouds or dedicated infrastructures



What is OpenStack ?





Getting started

1

Hosted private cloud

Full experience of a private cloud without in house infrastructure.

Public clouds

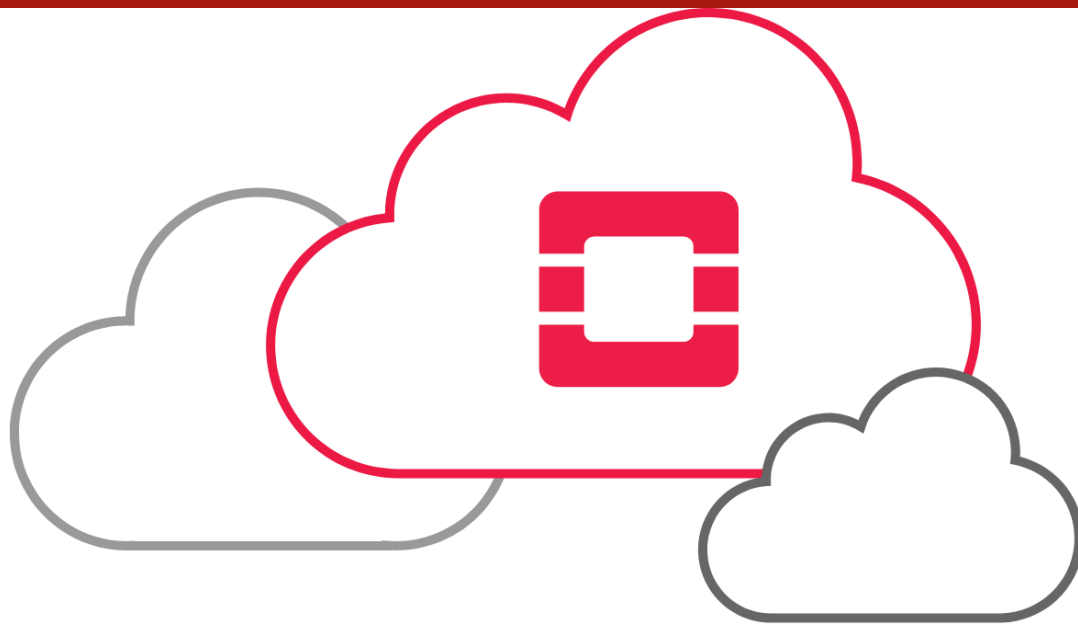
Get access to compute, storage and networking in minutes and only pay for what you use.





Microstack

OpenStack for micro clouds



```
$ sudo snap install microstack --beta --devmode|
```

```
$ sudo microstack init --auto --control|
```

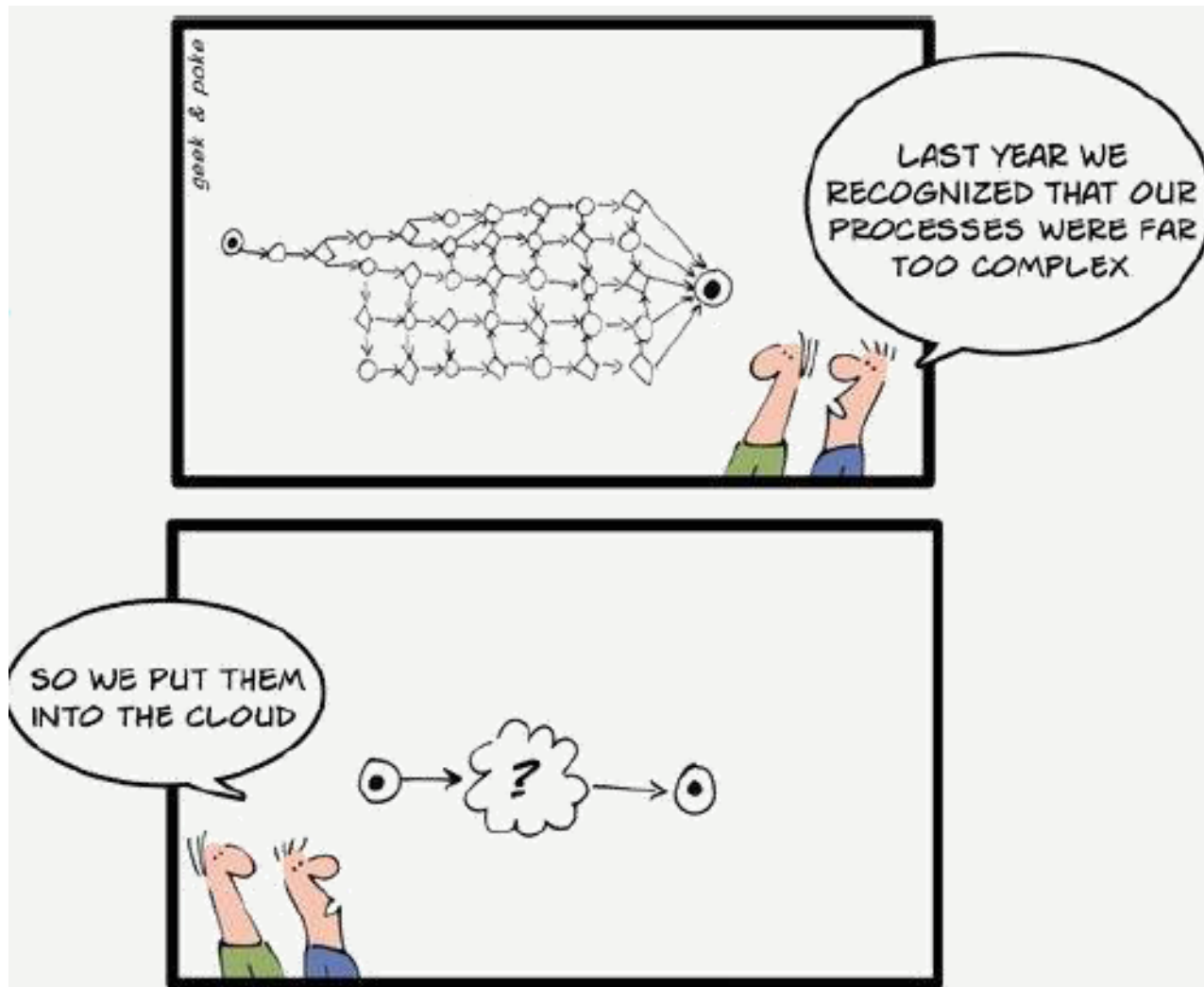

So... what's cloud computing?

- In some sense, the term means nothing!
 - If you make “full use” of modern off-the-shelf computing products and systems, you are a cloud computing user
 - You can't really buy “non-cloud” systems anymore

The Internet and cloud standards are built into everything

- You can block some features, but it is surprisingly hard to create a cloud-free computing system (one of many reasons it is so easy to break into many systems)

Cloud computing



Foolish Tech Prediction

"I think there is a world market for maybe five computers."
Thomas Watson, president of IBM, 1943



"computer ... vacuum-tube-powered
adding machine that's as big as a
house."

